

# UE-INF2320M Codage, Transmission, Compression D'Image Et De Vidéos



Université Claude Bernard



Lyon 1

## COTTIER Alexandre

*J'ai fait le choix pour ce TP de tout séparer en fichier différent, ça peut paraître très sous-efficient étant donné qu'il n'y a pas grand-chose d'écrit dans chaque fichier mais c'est une partie prise que j'assume car c'est pour moi un bon entraînement de séparer les fichiers car j'ai trop tendance à vouloir tout empaqueter dans un seul, et puis au niveau des tests c'était assez pratique de juste me débrouiller avec les commandes à lancer plutôt que de le bidouiller dans le code, j'ai essayé de rendre le code le plus optimisé et lisible possible en essayant d'appliquer les conseils que vous nous avez donnés lors des corrections de TP, j'ai aussi fait le choix de faire ce TP en langage C (je ne savais pas si c'était imposé ou non mais cela n'aurait pas modifié mon choix).*

*Scripts en C dans l'archive ainsi que des fichiers de test.txt*

## **1. Cœur de l'algorithme**

### **Direct : Transformation de Burrows-Wheeler**

L'algorithme réorganise les octets pour regrouper les caractères similaires sans réduire la taille, là est tout l'intérêt.

Son fonctionnement (j'ai voulu faire le plus simple possible) est qu'il trie les rotations cycliques du bloc d'entrée, le tri est effectué sur un tableau d'indices avec **qsort**, avec ça j'évite du coup les structures de données plus complexes.

En sortie on a donc :

- La taille totale du bloc (nombre d'octets).
- L'index permettant de retrouver la chaîne originale dans la matrice triée (numéro de ligne initiale).
- La suite d'octets transformée (dernière colonne).

### **Inverse : Reconstruction**

Pour l'inverse j'utilise la dernière colonne et le numéro de ligne pour reconstruire la première colonne (par tri), ensuite je remonte le fil des caractères pour retrouver exactement le fichier initial (compression sans perte).

## **2. Move To Front (direct et inverse)**

On a une table de 256 caractères, chaque caractère est remplacé par son index dans la table, puis déplacé en position 0.

Les suites de caractères identiques issues de la BWT sont transformées en une suite de zéros (index 0). Le format de sortie reste identique au flux d'entrée (suite d'octets).

## **3. RLE et Égalisateur de probabilité**

### **RLE**

Ce RLE là est un peu différent d'un RLE un peu plus classique vu qu'il est fait dans un contexte bien précis ou on veut que ce soit optimiser post-MTF:

- Seules les suites de 0 sont compressées. Une suite est remplacée par l'identifiant 0 suivi d'un octet représentant le nombre de répétitions.
- C'est à cette étape que la taille du fichier chute sur les fichiers structurés.

### **Égalisateur de probabilité**

En sortie de RLE, les données ne sont pas uniformes. J'utilise alors un égalisateur pour mesurer la distribution.

- Les tests montrent une concentration massive sur l'octet 0.
- L'entropie mesurée (3.267 bits/octet) signifie qu'un codage statistique final (type Huffman) permettrait d'atteindre la limite théorique de compression du bloc.

## TESTS

La chaîne de traitement utilisée est : **BWT → MTF → RLE**, j'ai fait 3 types de fichiers, un **répétitif** (motif ABCDE 2000 fois), un **normal** (j'ai juste copier-coller le code C d'un des script), et **binaire** (données aléatoires).

### Fichier Répétitif :

Taille initiale (octets) : 10 001      Taille finale : **26**      Taux de compression : **99,74%**

### Fichier Normal :

Taille initiale (octets) : 947      Taille finale : 784      Taux de compression : 17,21%

### Fichier Binaire :

Taille initiale (octets) : 10 000      Taille finale : **10 061**      Taux de compression : **-0,61%**

L'algorithme excelle sur les données hautement répétitives et l'expansion que l'on voit sur le fichier binaire s'explique par l'absence de redondance exploitable et le surcoût (overhead) des métadonnées du bloc BWT (8 octets).

## Et par rapport à l'ordre ?

Pour voir la pertinence du pipeline, je vais modifier l'ordre des filtres sur le fichier de test répétitif :

Optimale      **BWT → MTF → RLE**      Taille final : **26 o**

Sans MTF      **BWT → RLE**      Taille final : **10 009 o**

*Le RLE ne trouve aucun 0 à compresser.*

Inversée      **RLE → MTF → BWT**      Taille final : **10 009 o**

*La BWT ne peut plus réorganiser un flux déjà traité.*

**Pour le test 2 :** Même si la BWT regroupe les 'A', les 'B', etc., le RLE ne compresse que les zéros, sans le MTF pour transformer ces lettres en l'index 0, le RLE ne voit aucune suite de zéros et ne peut rien faire, l'augmentation (10 009) vient des 8 octets de métadonnées ajoutés par la BWT.

**Et pour le test 3 :** Si on lance la BWT à la fin, elle ne sert plus à rien pour aider la compression, la BWT doit être le premier filtre car son rôle est de créer la structure statistique que les filtres suivants vont exploiter.

Le Move-To-Front est indispensable entre la BWT et le RLE, c'est comme un traducteur qui convertit la « proximité » des caractères trouvés par la BWT en abondance de zéros pour le RLE.

## Par rapport à l'égalisateur

On a un aperçu de la structure statistique du flux sortant du RLE pour le fichier répétitif :

- Entropie mesurée : 3,267 bits/octet.
- L'octet 0 représente 26,9% du message.
- La taille minimale pourrait descendre à 11 octets avec un codage de Huffman

Un octet non compressé contient 8 bits d'information, mon pipeline a réduit cette incertitude à 3.267 bits.

L'égalisateur m'indique la taille théorique minimale de 11 octets, signifiant que si j'ajoutais un codage de Huffman (qui tasse les bits), on passerait de 26 octets actuels à seulement 11.

```
--- ANALYSE DE L'ÉGALISATEUR ---
Octet  0 : 26.9% des données
Octet  1 :  7.7% des données
Octet  2 :  7.7% des données
Octet 10 :  3.8% des données
Octet 14 :  3.8% des données
Octet 17 :  3.8% des données
Octet 39 :  3.8% des données
Octet 67 :  3.8% des données
Octet 68 :  3.8% des données
Octet 69 :  3.8% des données
Octet 70 :  7.7% des données
Octet 207 : 19.2% des données
Octet 208 :  3.8% des données
-----
Entropie : 3.267 bits/octet
Taille théorique minimale : 11 octets
-----
```

## **Conclusion**

L'implémentation que j'ai faite génère bien un flux contenant la taille (4 octets), l'index de ligne (4 octets) et la colonne transformée.

Le test de réversibilité avec la commande **diff** sur mon fichier de test *test\_normal.txt* montre bien que l'inverse reconstruit le fichier au bit près.

Par rapport au MTF, comme le montre le test 2 (sans MTF), sans lui, le RLE est inefficace. Il transforme la cohérence spatiale de la BWT en cohérence statistique (abondance de zéros).

Le RLE ne cible que les suites de zéros, sur *test\_repetitif.txt*, il permet de passer de 10 000 octets à 26 octets, cela montre bien l'efficacité radicale de cette méthode sur des motifs cycliques.

## **Commandes pour expérimenté :**

*Le ZIP du projet contient déjà des fichiers tout prêts pour les tests !*

Pour la compilation :

```
make clean
```

```
make
```

Pour compresser un fichier vers un format compressé :

```
cat document.txt | ./bwt | ./mtf | ./rle > document.comp
```

Pour restaurer le fichier original à partir du fichier compressé :

```
cat document.comp | ./irle | ./imtf | ./ibwt > document.res
```

Pour confirmer que la décompression est identique au bit près à l'original :

```
diff document.txt document.res
```

Cette commande permet de visualiser la distribution des octets et l'entropie finale du bloc compressé :

```
cat document.txt | ./bwt | ./mtf | ./rle | ./equalizer
```

Ces commandes permettent de mesurer l'impact de l'ordre des filtres sur la taille finale:

Mesure de la compression standard :

```
cat document.txt | ./bwt | ./mtf | ./rle | wc -c
```

Test sans le filtre Move-To-Front (MTF) :

```
cat document.txt | ./bwt | ./rle | wc -c
```

Test avec un ordre incohérent (BWT à la fin) :

```
cat document.txt | ./rle | ./mtf | ./bwt | wc -c
```